

1. Introduction

1.1 Program Overview CUSTPROC is a mainframe COBOL program developed by ANTIGRAVITY. The program interacts with various IBM data sources such as DB2 tables, IMS databases, VSAM files, IBM MQ queues, and CICS screens to perform complex database operations and manage corporate applications.

1.2 Objectives The objectives of the program are as follows: - To facilitate efficient storage, retrieval, and manipulation of large amounts of corporate information using IBM DB2. - To enhance data security through the use of IBM's IBM Security architecture. - To optimize transaction processing with the use of SQL features integrated into the program.

1.3 Scope The scope of the program involves interacting with multiple data sources such as: - Database tables in DB2 systems. - IMS databases and files, including VSAM archives. - IBM MQ queues for distributed transactional messaging. - CICS screens used for application scripting.

1.4 Assumptions and Constraints - The core functionalities of the program are fully supported by the given data sources. - No external requirements or conditions that would limit the scope of the program's interaction with these sources are anticipated or included in this description. - The program complies with IBM DB2, IMS database standards, and other relevant IBM software applications to ensure seamless integration.

In summary, CUSTPROC is a comprehensive mainframe COBOL program designed to integrate with various IBM data sources for efficient database operations and application management.

2. Database Details

2.1 DB2 Tables

The program queries or updates the following database tables: - DB2_CUST_TABLE

2.2 IMS Segments

The program interfaces with IMS databases using the following segment search arguments (SSAs) or IO areas: - Segment/IO Area: CUST-SEG-IO-AREA - Segment/IO Area: CUST-SEG-SSA

2.3 IDMS Records

The program retrieves or modifies the following IDMS database records: - Record:
CUST-RECORD-IDMS

3. System Architecture

3.1 Component Diagram

The diagram below outlines the external interfaces and components interfacing with the CUSTPROC program:

```
graph TD
    subgraph Mainframe_System [Mainframe System]
        PROG[CUSTPROC Program]
    end
    PROG -->|DB2 SQL| DB2_1[(DB2_CUST_TABLE Table)]
    PROG -->|VSAM I/O| FILE_2["CUSTFILE (VSAM-CUST-FILE)"]
    PROG -->|MQ Messaging| MQ[IBM MQ Queue Manager]
    PROG -->|CICS Map| CICS_3["Screen: CUSTMAP"]
    end
```

3.2 Control Flow Diagram

The diagram below shows the control flow and call relationships between the internal paragraphs of the program:

```
graph TD
    PROC-ENTRY["PROC-ENTRY"] --> B000-INITIALIZATION["B000-INITIALIZATION"]
    B000-INITIALIZATION --> C000-READ-CUSTOMER-DATA["C000-READ-CUSTOMER-DATA"]
    C000-READ-CUSTOMER-DATA --> D000-PROCESS-PII-DATA["D000-PROCESS-PII-DATA"]
    D000-PROCESS-PII-DATA --> E000-LOG-TO-MQ["E000-LOG-TO-MQ"]
    E000-LOG-TO-MQ --> F000-SEND-CICS-MAP["F000-SEND-CICS-MAP"]
    F000-SEND-CICS-MAP --> G000-HANDLE-ERROR["G000-HANDLE-ERROR"]
    G000-HANDLE-ERROR --> H000-TERMINATION["H000-TERMINATION"]
    H000-TERMINATION --> X999-DEAD-PARAGRAPH["X999-DEAD-PARAGRAPH (Dead Code)"]
    X999-DEAD-PARAGRAPH --> PROC-ENTRY
    PROC-ENTRY --> B000-INITIALIZATION
    PROC-ENTRY --> C000-READ-CUSTOMER-DATA
    PROC-ENTRY --> D000-PROCESS-PII-DATA
    PROC-ENTRY --> E000-LOG-TO-MQ
    PROC-ENTRY --> F000-SEND-CICS-MAP
    PROC-ENTRY --> G000-HANDLE-ERROR
    PROC-ENTRY --> H000-TERMINATION
    B000-INITIALIZATION --> G000-HANDLE-ERROR
    C000-READ-CUSTOMER-DATA --> G000-HANDLE-ERROR
    E000-LOG-TO-MQ --> G000-HANDLE-ERROR
    G000-HANDLE-ERROR --> G000-HANDLE-ERROR
    classDef dead fill:#fbcfe8,stroke:#f43f5e,stroke-width:2px;
    class PROC-ENTRY, B000-INITIALIZATION, C000-READ-CUSTOMER-DATA, D000-PROCESS-PII-DATA, E000-LOG-TO-MQ, F000-SEND-CICS-MAP, G000-HANDLE-ERROR, H000-TERMINATION, X999-DEAD-PARAGRAPH dead;
```

4. Detailed Design

4.1 Program Structure

The following table details all sections and paragraphs defined in the program:

Paragraph / Section	Start Line	End Line	Section Group	Description
PROC-ENTRY	42	53	A000-MAIN-PROCESSING	Initial processing logic
B000-INITIALIZATION	54	63	A000-MAIN-PROCESSING	Initial processing logic
C000-READ-CUSTOMER DATA	64	89	A000-MAIN-PROCESSING	Queries DB2, Calls IMS, Accesses VSAM files
D000-PROCESS-PII-DATA	90	95	A000-MAIN-PROCESSING	Initial processing logic
E000-LOG-TO-MQ	96	107	A000-MAIN-PROCESSING	Outputs to IBM MQ
F000-SEND-CICS-MAP	108	116	A000-MAIN-PROCESSING	Makes CICS Maps
G000-HANDLE-ERROR	117	121	A000-MAIN-PROCESSING	Initial processing logic
H000-TERMINATION	122	124	A000-MAIN-PROCESSING	Initial processing logic
X999-DEAD-PARAGRAPH	125	128	A000-MAIN-PROCESSING	Initial processing logic

4.2 Algorithms

4.2.1 Overall Program Logic

The program begins by executing the CUSTPROC-ENTRY procedure. This procedure is structured into lines numbered from 42 to 53, which call each other through lines 54 and 63. The overall logic of the program involves:

1. **Initialization:**
2. Lines 54 and 63 are marked as G000-INITIALIZATION.
3. This indicates that a DB2 procedure is being called to handle any database initialization errors.
Lines 64 to 89 contain calls to the same G000-INITIALIZATION procedure with an empty argument list, indicating no specific error handling details are provided.

File/Database Reading:

6. Lines 90 and 95 call a DB2 procedure with an empty argument list to read customer data from a file.
 - DB2 calls:: This indicates that this step is executed without any specific errors or messages.

Line 107 also contains calls to the same DB2 procedure, but in this context, it likely represents a check for error codes returned by the process.

Processing PII Data:

Lines 96 and 108 both call the G000-PROCESS-PII-DATA procedure with empty argument lists.

- These lines represent steps to process customer data, possibly performing operations on it or sending it to a specific destination.

Logging to MQ:

Lines 97 and 116 contain calls to the same G000-LOG-TO-MQ procedure with empty argument lists.

- These lines represent steps to log messages to a Message Queue (MQ), possibly for monitoring or reporting purposes.

Sending CICS Map:

Lines 98 and 117 contain calls to the same G000-SEND-CICS-MAP procedure with empty argument lists.

- These lines represent steps to send data from a customer service process (CICS) to a specific destination.

Handling Error Messages:

Lines 120 and 121 contain calls to the same G000 - HANDLE - ERROR procedure, which handles any errors or messages generated during processing.

- These lines represent steps where error handling logic is executed for specific scenarios or data formats.

4.2.2 Key Algorithmic Details

The program's overall algorithm involves a series of steps that are structured as follows:

1. Initialization:

The CUSTPROC -ENTRY procedure calls a DB2 procedure (lines 54 and 63) to handle any database initialization errors.

- `DB2 calls`: This indicates the general error handling logic associated with the database initialization, likely covering issues like syntax errors or incompatibilities.

File/Database Reading:

The next step involves reading data from a file (line 90) and passing it to the same DB2 procedure (lines 64-89) for further processing.

- `DB2 calls`: This indicates that this call is part of general error handling, covering common issues like file I/O errors.

Processing PII Data:

The next step involves reading customer data from a specific database or table (lines 96-107) and passing it to the same DB2 procedure for processing.

- `DB2 calls`: This indicates that this call is part of general error handling, covering common issues like data integrity errors.

Logging to MQ:

The program logs any errors or messages generated during the processing steps (lines 97 and 108) into a Message Queue (MQ) (lines 96-107).

- `DB2 calls`: This indicates that this call is part of general error handling, covering common issues like connection errors.

Sending CICS Map:

The program sends data from the customer service process to a specific destination (lines 98 and 116) and processes it with an IMS procedure.

- `DB2 calls`: This indicates that this call is part of general error handling, covering common issues like database connectivity or communication errors.

Handling Error Messages:

12. The program handles any specific errors or messages generated during the processing steps (lines 120-121) with an IMS procedure.
 - **DB2 calls:** This indicates that this call is part of general error handling, covering common issues like database connectivity problems.

4.2.3 Expected Outcome

The program aims to: - Ensure the customer data is read and processed without any errors. - Log messages or exceptions into a Message Queue (MQ) for further analysis. - Send data from the customer service process to a specific destination using IMS procedures. - Handle any issues that arise during the processing steps with an error-handling function in DB2.

4.3 Input/Output Specifications

The primary files and UI interfaces are defined below:

Interface Channel	Technical Resource / DD Name	Description	Data Format	Direction
VSAM File	VSAM-CUST-FILE	File variable CUSTFILE	Index Keyed Record	In/Out
IBM MQ	MQ Series Queue	Buffer message transmission	MQ Descriptor Block	Output
CICS Terminal	CUSTMAP Screen	Terminal user-interface map	Screen Layout Fields	Output

4.4 DB2 Database Details

- **Paragraph C000-READ-CUSTOMER-DATA:** Executes SELECT on table DB2_CUST_TABLE. SQL Query: `sql SELECT CUST_NAME, CUST_SSN, CUST_ACCOUNT_NUM INTO :CUST-NAME, :CUST-SSN, :CUST-ACCOUNT-NUM FROM DB2_CUST_TABLE WHERE CUST_ID = :CUST-ID`

4.5 IMS Database Details

- **Paragraph C000-READ-CUSTOMER-DATA:** Invokes IMS command GU using parameters:
CUST-SEG-IO-AREA, CUST-SEG-SSA

4.6 IDMS Database Details

- **Paragraph C000-READ-CUSTOMER-DATA:** Executes IDMS OBTAIN on record
CUST-RECORD-IDMS within Area CUST-AREA-IDMS

4.7 Called Sub-routine/Program Details/Copybook

No copybooks or external subprograms referenced.

4.8 VSAM File Details

- **Paragraph C000-READ-CUSTOMER-DATA:** Performs READ on file variable CUSTFILE
(Dataset/DD: VSAM-CUST-FILE)

4.9 IBM MQ Details

- **Paragraph E000-LOG-TO-MQ:** Executes IBM MQ function MQPUT with argument trace:
MQ-HCONN, MQ-HOBJ, MQ-MD, MQ-PMO, WS-MQ-MSG-BUFF, MQ-REASON-CODE

4.10 CICS Details

- **Paragraph F000-SEND-CICS-MAP:** Runs SEND MAP('CUSTMAP') on map CUSTMAP (Mapset:
CUSTSET). CICS details: SEND MAP('CUSTMAP') MAPSET('CUSTSET')
FROM(CUSTOMER-RECORD) ERASE

4.11 Error Handling

- **Error Paragraph G000-HANDLE-ERROR:** Handles faults by logging or displaying errors. Code snippet: ``cobol DISPLAY 'ERROR OCCURRED. SQL-CODE=' SQL-CODE-VAL DISPLAY 'VSAM-STATUS=' VSAM-STATUS MOVE 'AN ERROR WAS ENCOUNTERED IN PROCESSING' TO ERR-MSG.

``

5. Interface Design

5.1 External Interfaces

This COBOL program communicates with external systems, protocols, and the CICS user interface.

DB2 Tables (DB2_CUST_TABLE)

The COBOL program accesses the `DB2_CUST_TABLE` table to retrieve data related to customers.

VSAM Files

- **VSAM-CUST-FILE:** The COBOL program accesses this file using a dataset named `VSAM-CUST-FILE`. This indicates that the dataset is accessible from outside the database system, allowing for external interactions with the CICS user interface and other systems.

6. Testing Strategy

6.1 Test Plan

To ensure the reliability, integrity, and functionality of the mainframe COBOL program 'CUSTPROC', a comprehensive testing strategy is required. This test plan outlines the steps to be taken during the various phases of the development process.

6.1.1 Unit Testing

- **Objective:** Verify that individual components and modules function as expected without relying on external dependencies.
- **Methodology:**
 - Define the unit tests by structuring the program into functions, each testing a specific part or module.
 - Write test cases using COBOL code that perform basic operations such as reading data from files, interacting with databases, and performing simple queries.

6.1.2 Database Stubbing/Mocking

- **Objective:** Simulate database access to allow for more controlled integration tests without exposing the full functionality of the real database system.
- **Methodology:**
 - Create a stub environment where mock data can be inserted into the COBOL program.
 - Write test cases that use the mock data and assert the expected results from the stubbed function or method.

6.1.3 Online CICS Transaction Testing

- **Objective:** Verify that transactions in the mainframe system interact correctly with the COBOL program without any external interruptions.
- **Methodology:**
 - Use an online transaction processing environment (CICS) to simulate real-world interactions between the mainframe and COBOL programs.
 - Write test cases that simulate different scenarios, such as inserting data into tables in the database, updating records in a database table, and verifying transactions within the mainframe system.

6.1.4 CICS MAP Interface Testing

- **Objective:** Evaluate the integration between the mainframe COBOL program 'CUSTPROC' and the CICS MAP interface.
- **Methodology:**
- Use CICS MAP interface to simulate interactions with other systems or databases outside of the COBOL program.
- Write test cases that involve interaction with various CICS components, such as interacting with a database, performing transactions in a transaction log, and managing system events.

6.1.5 Testing Environment Setup

- **Objective:** Ensure all testing tools are available during the development process.
- **Methodology:**
- Install development tools (COBOL compiler, COBOL editor, DB2 SQL client) for testing purposes.
- Use testing frameworks and libraries specific to the environment (e.g., JUnit, Selenium for web applications).

6.1.6 Continuous Integration/Continuous Deployment (CI/CD)

- **Objective:** Automate tests to ensure that changes made during development are integrated into production without manual intervention.
- **Methodology:**
- Set up a CI/CD pipeline that integrates with testing tools and deploys the latest code regularly.
- Configure automated builds and tests to run daily, weekly, or monthly.

6.1.7 Documentation and Reporting

- **Objective:** Maintain a comprehensive set of test reports and metrics for monitoring the program's performance and effectiveness.
- **Methodology:**
- Write detailed test scripts that capture various aspects of the program (e.g., unit tests, database operations, transaction failures).
- Use test harnesses like JUnit or Python to automate and collect data from the CICS system during testing.

6.1.8 Stakeholder Management

- **Objective:** Ensure that all stakeholders are aware of the testing strategy and its implications.
- **Methodology:**

- Share detailed information about the testing process with end-users, developers, and other relevant parties.
- Provide documentation on how to access and manage test results during the development cycle.

6.2 Testing Environment

6.2.1 COBOL Compiler and Editor

- **Objective:** Compile and run the mainframe COBOL program 'CUSTPROC' using a COBOL compiler tool.
- **Methodology:**
- Install and configure the COBOL development environment (e.g., CobolWorks) to compile and execute the COBOL code.

6.2.2 COBOL Editor

- **Objective:** Edit, modify, and interact with the mainframe COBOL program 'CUSTPROC'.
- **Methodology:**
- Use a COBOL editor for editing specific sections or functionalities of the program.
- Test edit operations through interactive testing cases.

6.2.3 Database Interaction

- **Objective:** Simulate database interactions using an online transaction processing environment (CICS).
- **Methodology:**
- Connect to the CICS system and perform data retrieval, update, or insertions within the program 'CUSTPROC'.
- Write test cases that mock database operations to simulate real-world scenarios.

6.2.4 Transaction Management

- **Objective:** Test transactional behaviors in the mainframe system.
- **Methodology:**
- Insert and retrieve data from tables in a simulated database using COBOL transactions within the program 'CUSTPROC'.
- Verify that all expected data retrieval operations are successful.

6.2.5 CICS MAP Interface

- **Objective:** Simulate interactions between the mainframe COBOL program 'CUSTPROC' and a transaction log in the CICS system.
- **Methodology:**
- Use an online transaction processing environment (CICS) to simulate real-world transactions and events within the mainframe system.
- Write test cases that interact with CICS components, such as verifying data updates, log entries, or database access.

6.2.6 CI/CD Pipeline

- **Objective:** Automate testing processes by integrating them into a continuous integration and deployment (CI/CD) pipeline.
- **Methodology:**
- Configure the CI/CD pipeline with Jenkins or any other tool that supports automated tests and deployments.
- Build and test the mainframe COBOL program 'CUSTPROC' using the latest code changes every time they are deployed.

6.2.7 Testing Metrics

- **Objective:** Monitor the effectiveness of testing by logging and analyzing test results during development.
- **Methodology:**
- Set up metrics tracking such as error rates, test coverage, and defect rate.
- Use the logs and reports to identify patterns or areas for improvement.

6.2.8 User Documentation

- **Objective:** Provide users with clear instructions on how to access and use COBOL programs during the testing process.
- **Methodology:**
- Write user guides or step-by-step documentation that include detailed explanations of each test case's functionality.
- Ensure that all necessary resources, such as database tables, are easily accessible.

6.2.9 Collaboration

- **Objective:** Facilitate collaboration among development teams and stakeholders during the testing process.
- **Methodology:**

- Hold meetings or workshops to discuss test results and identify areas for improvement.
- Share best practices and common pitfalls with the entire team.

6.3 Tools List

1. COBOL Compiler: A software application that compiles COBOL programs into machine code.
2. COBOL Editor: Software tool for editing COBOL programs.
3. Database interaction tools (e.g., CICS, IMS):
4. COBOL Database Interface (COBOL DIA) or COBOL Data Access Interface (CDA).
5. Database management systems that interact with the mainframe system.
6. Transaction management:
7. COBOL transactions for database interactions.
8. CICS transaction log and simulation tools: e.g., CICS Simulation and Testing Environment (CSTAR) tool.
9. Continuous integration/continuous deployment (CI/CD) pipeline: Tools like Jenkins, GitLab CI, or GitLab Runner.
10. User documentation tools: e.g., Microsoft Word with templates, PDFs, or JIRA for collaborative writing and approval processes.
11. Monitoring and logging systems: e.g., Nagios, Splunk for monitoring network performance and application logs.

6.4 Dependencies

6.4.1 COBOL Compiler

- **Dependencies:** CobolWorks, a free open-source development environment that supports compiling COBOL programs into machine code.
- **Usage:** Use the CobolWorks compiler to compile the mainframe COBOL program 'CUSTPROC'.
- **Impact:** Helps ensure that the COBOL code runs correctly on the mainframe and can be executed during testing.

6.4.2 COBOL Editor

- **Dependencies:** Any COBOL editor tool.
- **Usage:** Use the COBOL editor to edit, modify, and interact with the mainframe COBOL program 'CUSTPROC'.
- **Impact:** Helps maintain code quality and readability during development.

6.4.3 Database Interaction Tools (e.g., CICS)

- **Dependencies:** COBOL DIA or CDA, database management systems that connect to the mainframe system.
- **Usage:** Use COBOL DIA or CDA for interacting with databases in a simulated manner within 'CUSTPROC'.
- **Impact:** Helps ensure data integrity and manage data access permissions.

6.4.4 Transaction Management

- **Dependencies:** Database transactions, transaction management tools for the mainframe system.
- **Usage:** Use database transactions to simulate real-world database operations inside 'CUSTPROC'.
- **Impact:** Ensures correct database interactions and prevents unexpected side effects.

6.4.5 CICS MAP Interface

- **Dependencies:** Online transaction processing environment (CICS) that interacts with the mainframe system.
- **Usage:** Use CICS simulation tools to simulate real-world transactions within 'CUSTPROC'.
- **Impact:** Helps ensure data integrity and correct execution of external system events.

6.4.6 CI/CD Pipeline

- **Dependencies:** Jenkins, GitLab CI, or any other CI/CD tool.
- **Usage:** Use a CI/CD pipeline to automate testing processes by integrating them into the development process.
- **Impact:** Ensures that changes in 'CUSTPROC' are tested before integration with the mainframe system.

6.4.7 User Documentation Tools

- **Dependencies:** Microsoft Word, PDFs, or JIRA for collaborative writing and approval processes.
- **Usage:** Share user documentation documents such as user guides and step-by-step instructions during testing.
- **Impact:** Facilitates easy access to detailed information about the program's functionality and interactions with external systems.

By following this comprehensive test plan, unit testing, database stubbing/mocking, online CICS transaction testing, CICS MAP interface testing, continuous integration/continuous deployment (CI/CD) pipeline, documentation, stakeholder management, CI/CD infrastructure setup, user documentation tools, and collaboration among teams, you can effectively verify the reliability and functionality of 'CUSTPROC' throughout its entire development lifecycle.

7. Performance Considerations

7.1 Performance Analysis

To evaluate the performance of the COBOL program named 'CUSTPROC', we need to consider several key factors: latency, CPU utilization (MIPS), database indexing, buffer sizes, and optimal host variable patterns.

Latency

Latency is typically measured by the time it takes for a program to process data. In this context, it refers to how long it takes for a COBOL program to execute multiple SELECT statements from DB2 SQL SELECT, IMS database reads, and VSAM index file reads.

- **Database Indexing:** The effectiveness of the database indexing is crucial in improving query performance. If the database indexes are poorly designed or not optimized, they can slow down queries.
- **Optimization Recommendations:** To optimize database indexing: `sql SELECT * FROM CUSTOMERS WHERE CUST_ID IN (SELECT DISTINCT CUSTOMER_ID FROM CUSTOMERS WHERE CUSTOMER_NAME = 'Example Customer')`

Additional Measures: Implementing indexes on frequently queried columns can significantly improve performance.

Database Performance Metrics: To monitor the latency, we might use tools like `ps` for processes and `iostat` to measure disk I/O. For example: `sql SELECT * FROM CUSTOMERS WHERE CUST_ID IN (SELECT DISTINCT CUSTOMER_ID FROM CUSTOMERS WHERE CUSTOMER_NAME = 'Example Customer')`

CPU Utilization (MIPS)

CPU utilization is the ratio of the amount of time that a program spends running to the total available system cycles. A high MIPS indicates that the CPU is being used more efficiently.

- **Optimization Recommendations:** To optimize CPU usage: `sql SELECT * FROM CUSTOMERS WHERE CUST_ID IN (SELECT DISTINCT CUSTOMER_ID FROM CUSTOMERS WHERE CUSTOMER_NAME = 'Example Customer')` This query will return results faster by not waiting for the VSAM index to be read.

Database Indexing

Database indexing is crucial for efficient database queries. If the indexes are sparse, they can slow down data access times.

- **Optimization Recommendations:** To optimize database indexing: `sql SELECT * FROM CUSTOMERS WHERE CUST_ID IN (SELECT DISTINCT CUSTOMER_ID FROM CUSTOMERS WHERE CUSTOMER_NAME = 'Example Customer')`

Buffer Sizes

Buffer sizes refer to the amount of memory used by a program to buffer and store data. Larger buffer sizes can improve performance, but they also require more system resources.

- **Optimization Recommendations:** To optimize buffer usage: `sql SELECT * FROM CUSTOMERS WHERE CUST_ID IN (SELECT DISTINCT CUSTOMER_ID FROM CUSTOMERS WHERE CUSTOMER_NAME = 'Example Customer')`

Optimal Host Variable Patterns

The optimal host variable pattern is the arrangement of variables that minimizes latency and improves performance. This often involves using arrays and other data structures to store frequently accessed data.

- **Optimization Recommendations:** To optimize the host variable pattern: `sql SELECT * FROM CUSTOMERS WHERE CUST_ID IN (SELECT DISTINCT CUSTOMER_ID FROM CUSTOMERS WHERE CUSTOMER_NAME = 'Example Customer')`

Conclusion

To summarize, the performance of the COBOL program 'CUSTPROC' can be analyzed through these metrics. Monitoring and adjusting the database indexing strategy to reduce query time is particularly effective here.

```
# 7.2 Optimization Recommendations Based on our analysis, the optimal
combination of latency, CPU utilization (MIPS), database indexing, buffer sizes,
and host variable patterns would optimize the program's performance: -
**Latency:** Implementing sparse indexes can significantly improve query
performance. - **CPU Utilization (MIPS):** Optimize data access by using
appropriate database indexes and avoiding frequent reads from the VSAM index
file. - **Database Indexing:** Selectively use indexes for frequently queried
columns to minimize disk I/O. - **Buffer Sizes:** Minimize the amount of memory
used per query by optimizing variable types and reducing buffer sizes. ###
Additional Measures - **Optimization Techniques:** Implement caching, minimizing
system calls, and using asynchronous operations if applicable. - **Schema
Changes:** Revisit the database schema for optimal performance improvements.
```sql ALTER TABLE CUSTOMERS ADD (VIP boolean); ```
```

## 8. Dead Code Analysis

---

The control flow reachability analysis has identified **1** unreachable paragraphs:

- `X999 - DEAD - PARAGRAPH`

**Risk/Impact:** These paragraphs represent dead code. Having dead code increases maintenance overhead, confuses developer comprehension, and can indicate orphaned logical blocks or legacy routines that are no longer in use. It is recommended to safely comment out or delete these blocks after verifying no dynamic/computed performance routes (e.g. dynamic CALLs) target them.

## 9. Security Risk Assessment

---

### 9.1 PII (Personally Identifiable Information) Analysis

---

#### 9.1.1 PII Data Elements Identification

The following variables contain or handle PII based on name pattern rules:

Variable Name	Level	PIC Clause	PII Classification Reason
WS-PII-LOG-SSN	5	X(9)	Declared definition

#### 9.1.2 PII Data Flow Analysis

Static taint tracking shows the propagation of PII data across variable reassignments:

| N/A | None | None | N/A | No PII flows observed |

## 9.1.3 PII Security Analysis

---

### Summary:

The program 'CUSTPROC' contains sensitive personal identification (PII) variables including SSN. This analysis focuses on the potential risks associated with unencrypted data transmissions and the use of logging to monitor for unauthorized access or exposure of this information.

### Encryption:

- **Encryption Type:**
- **Primary Source:** None explicitly stated, but it could be assumed that the program uses some form of encryption.  
**Alternative Sources:**
  - Data has been encrypted using an RACF (Real-time Access Control Forum) environment to prevent unauthorized access.
  - The SSN is being logged and monitored using a logging mechanism, which should include some form of encryption.

### Logs Exposure:

- **Logging Mechanism:**
- **Primary Source:** No external logs are listed; this indicates that the system does not have an explicit or implicit log retention policy.

#### Alternative Sources:

- The data has been logged to a security log in the RACF environment.
- Logs from the program's output can be monitored via the console.

#### Audit Trails:

- **Primary Source:** No external audit trails are noted; logs are being used to monitor user activity and ensure compliance with security policies.

#### Alternative Sources:

- The SSN data is not stored in a secure manner for this particular use.
- Logs from the program's output can be scrutinized by administrators.

### MQ Transmission Risks:

- **MQ Mechanism:**



- **Primary Source:** No explicit information about using an MQTT (Message Queue Transport Protocol) or similar protocol to transmit data is provided.

**Alternative Sources:**

- The data is being transmitted through the console or a communication interface, which can be monitored by administrators.

**Mitigation Strategies:**

- **Encryption:**

Encrypt sensitive data both at rest and in transit using strong encryption methods (e.g., AES).

**Logging:**

Implement logging mechanisms to track data usage and ensure compliance with security policies. Ensure logs are encrypted and accessible only for authorized personnel.

**Audit Trails:**

- Monitor user activity through audit trails that can be accessed by administrators.

**Conclusion:**

While this program does not contain PII variables, the use of unencrypted data transmission and logging mechanisms poses several risks. RACF or cryptography should be used to ensure secure communication between the client and the server, and logs should be encrypted for security purposes. Monitor user activity through audit trails for any unauthorized access attempts.

**Security Practices:**

- **RACF Environment:** Use an RACF environment to ensure data is securely stored and accessed.
- **Encryption:** Implement strong encryption methods for both storage and transmission of sensitive data.
- **Logging:** Implement logging mechanisms that can be monitored by administrators and used to identify potential security vulnerabilities.
- **Audit Trails:** Monitor user activity through audit trails for any unauthorized access attempts.

This analysis provides a comprehensive perspective on the risks associated with unencrypted data transmissions and the use of logging mechanisms in this context.

#### 9.1.4 PII Inventory Summary

A summary of PII egress points and risk classifications:

| N/A | None | Low | No PII transmission detected |

# 10. Appendices

---

## 10.1 Glossary

Term	Definition
<b>Copybook</b>	A reusable source file that contains data structures or declarations in COBOL.
<b>DB2</b>	IBM's relational database management system for z/OS mainframe environments.
<b>IMS</b>	Information Management System, a database and transaction management system.
<b>VSAM</b>	Virtual Storage Access Method, a high-performance file management system on IBM mainframes.
<b>IBM MQ</b>	Message queue middleware facilitating secure, asynchronous system-to-system communications.
<b>CICS</b>	Customer Information Control System, an online transaction manager for mainframe systems.
<b>PII</b>	Personally Identifiable Information; sensitive data elements that can identify individuals.
<b>Dead Code</b>	Program blocks or paragraphs that cannot be reached or executed under any conditions.

## 10.2 Data Structures

---

No complex layouts or copybooks imported.

## 10.3 References

---

- IBM Enterprise COBOL for z/OS Language Reference

- [DB2 SQL Reference Guide](#)
- [CICS Application Programming Guide](#)
- [IBM MQ Application Programming Reference](#)